and how you can significantly speed up your workflows

with help of caching.

Show less

This transcript explains **how dependency caching works in GitHub Actions** and how it improves workflow performance. Here's a simplified explanation:

Scenario 1: Running the Workflow Again (No Dependency Changes)

Suppose you:

1. Change a source file (e.g., change a heading from `<h3>` to `<h2>`).
2. Commit the change.
3. Push the commit.

When the GitHub Actions workflow runs again:

- The **cache key remains the same** because `package-lock.json` has not changed.
- GitHub Actions finds the previously saved cache.
- The **"Restore Cache"** step downloads the cached `node_modules` (or npm cache).
- `npm install` (or `npm ci`) finishes much faster because most packages don't need to be downloaded again.

Workflow

```
Previous Workflow
-----------------
npm ci
↓
Download all dependencies
↓
Save cache

Next Workflow
-------------
Restore cache ✅
↓
npm ci
↓
Only verify/install missing packages
```

```
↓
Build/Test (faster)
```

Why Every Job Becomes Faster

Even if your workflow has multiple jobs:

```
jobs:
  install:
  test:
  lint:
  build:
```

Each job can restore the same cache.

Instead of downloading packages repeatedly:

```
Job 1
Restore Cache ✅

Job 2
Restore Cache ✅

Job 3
Restore Cache ✅
```

This saves several minutes in large projects.

Scenario 2: Dependencies Change

Now suppose you update packages.

Bash

```bash
npm outdated
npm update
```

This changes:

```
package.json
package-lock.json
```

Git detects these files changed.

You commit:

```
git add .
git commit -m "Updated dependencies"
git push
```

What Happens to the Cache?

Most GitHub Actions cache examples use a key like:

```yaml
key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
```

The important part is:

```
hashFiles(package-lock.json)
```

When `package-lock.json` changes:

Old hash:

```
A45D8F
```

New hash:

```
C93EF2
```


Therefore:

Old cache key:

```
ubuntu-node-A45D8F
```

New cache key:

```
ubuntu-node-C93EF2
```

Since the key is different:

```
Cache Miss ❌
```

GitHub Actions cannot reuse the previous cache.

What Happens Next?

The workflow now does:

```
No cache found
↓
npm ci
↓
Download all packages
↓
Install dependencies
↓
Save new cache
```

After the workflow finishes:

```
Cache
ubuntu-node-C93EF2
```

Now every future workflow using the same lock file will use this cache.

Lifecycle of a Cache

```
Workflow #1

package-lock.json
Hash = ABC123

No Cache
    |
    ▼
Download Dependencies
    |
    ▼
Save Cache
(Cache ABC123)
```

↓

```
Workflow #2

package-lock.json
Hash = ABC123

Restore Cache ✅
    |
    ▼
npm ci (Very Fast)
```

↓

```
Developer updates dependencies
```

↓

```
package-lock.json
Hash = XYZ789
```

↓

```
Workflow #3

Restore Cache ❌
(Cache Key Changed)

Download Dependencies
    |
    ▼
Save New Cache
(Cache XYZ789)
```

↓

```
Workflow #4

Restore Cache (XYZ789) ✅
```

Visual Timeline

```
Initial Workflow
_____________________________

Cache Exists? ❌
```

```
Download Packages
Save Cache (Key A)

        |
        ▼

Next Commit (Source Code Only)
──────────────────────────────────────

Cache Exists?  ✅
Restore Cache
Install = Fast

        |
        ▼

Next Commit (Dependency Update)
──────────────────────────────────────

package-lock.json Changed

Cache Key = New

Cache Exists? ❌
Download Packages
Save Cache (Key B)

        |
        ▼

Future Commits
──────────────────────────────────────

Cache Exists? ✅
Restore Cache
Install = Fast
```

Typical GitHub Actions Configuration

YAML

```yaml
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-

- name: Install dependencies
  run: npm ci
```

- `path` specifies what to cache (here, the npm package cache).
- `key` is derived from the operating system and the hash of `package-lock.json`.
- If `package-lock.json` changes, the hash changes, creating a new cache automatically.
- `restore-keys` allows GitHub Actions to fall back to a less specific cache if an exact match isn't available.

Key Takeaways

- **Source code changes only** → cache is reused → faster installs.
- `package-lock.json` **changes** → new cache key → cache miss → dependencies are downloaded again.
- After the first run with the updated dependencies, the **new cache is saved**.
- All subsequent workflow runs with the same dependency versions benefit from the new cache, significantly reducing build times.